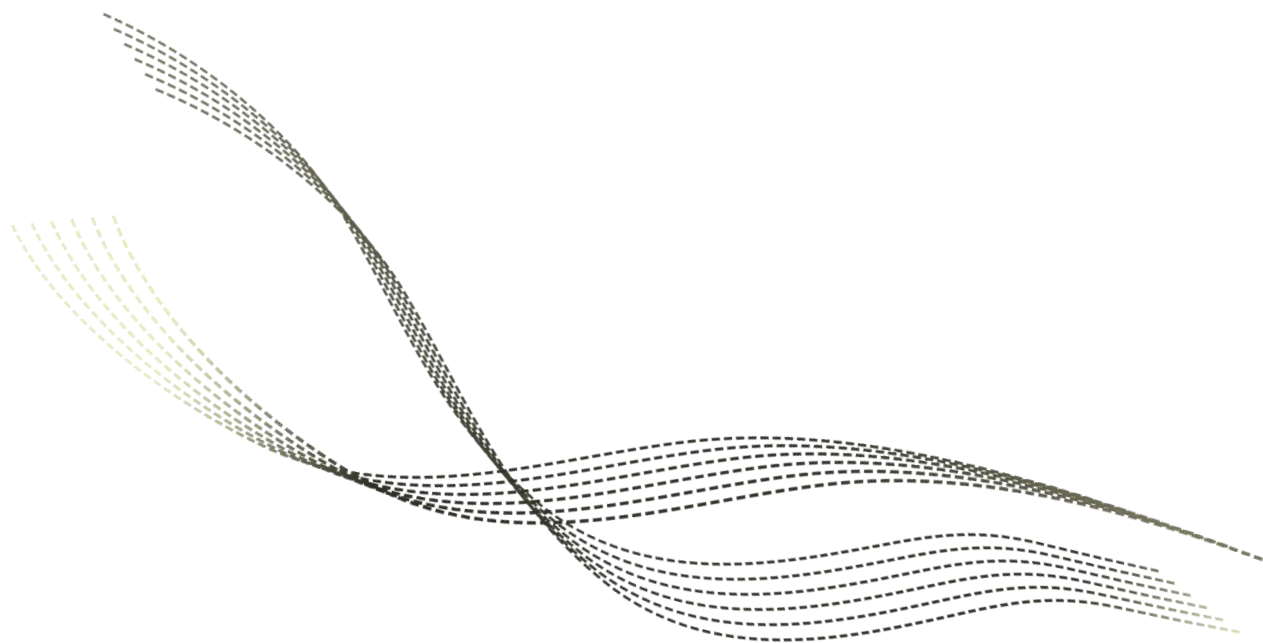


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路布局方法

沈明华



目录

CONTENTS

01

布局问题

02

模拟退火

03

图划分

04

解析式布局

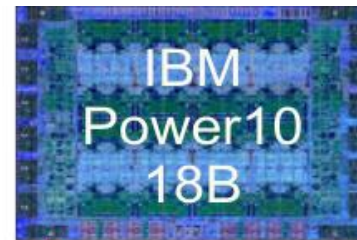
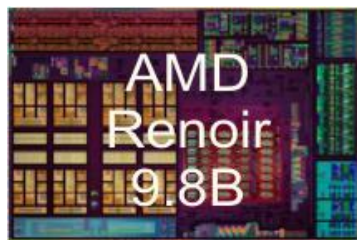
PART 01

什么是布局

■ 布局(Placement)

□ 背景

- 集成电路(芯片)技术已成为大国博弈的前沿
 - 百亿规模的EDA产业撑起了万亿规模的集成电路产业
- 现代芯片设计的挑战
 - 元件尺寸不断缩小，设计复杂度不断提高
 - 设计规则繁多，性能要求高



- **布局(placement)**是芯片设计流程中确定逻辑门物理位置的步骤
 - 影响芯片的时序、功耗、面积等
 - 对**百亿晶体管规模**的电路进行布局极具挑战性

■ 大纲

□ 什么是布局

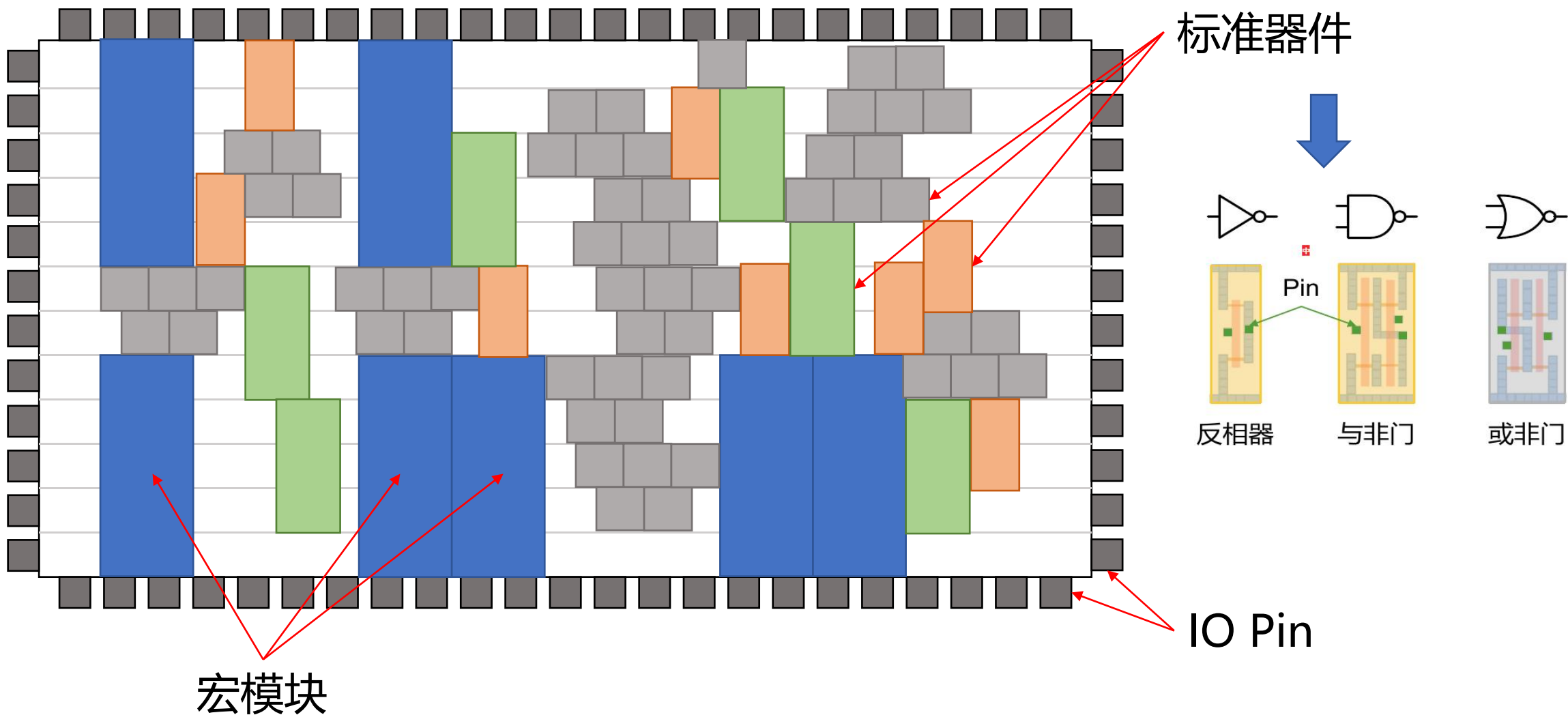
- 布局算法历史

□ 全局布局(Global placement)

- 模拟退火: Dragon2000 & Timberwolf
- 图划分: CAPO
- 二次函数优化布局: Gordian & SimPL
- 非线性布局: NTUPlace & ePlace

■ 什么是布局

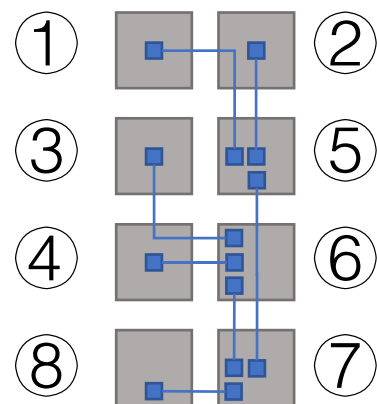
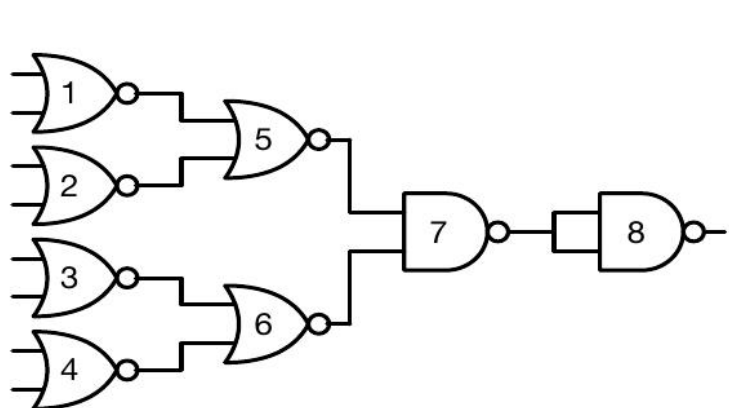
□ 布局就是把逻辑门放到芯片基板上的过程



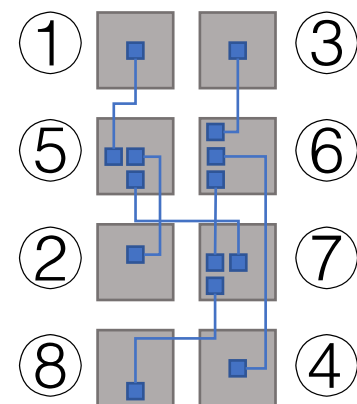
■ 什么是布局

□ 好的布局应该让相连模块之间的距离尽可能小

– 线长(wirelength): 连接模块物理导线的总长度



总线长: 100



总线长: 150

■ 什么是布局

□ 问题定义

– 输入

- 模块(包括标准器件和宏模块) $B_1, B_2, \dots, B_n$
- 每个模块的形状和引脚(pin)的位置
- 连接模块的导线(nets) $N_1, N_2, \dots, N_m$

– 输出

- 模块的坐标 (x, y)
- 要求在给定的区域内布局模块, 且没有模块重叠

– 优化目标

- 最小化布线的总线长
- 其他可能的附带目标: 时序、可路由性、延迟等等

■ 什么是布局

□ 布局问题的难度

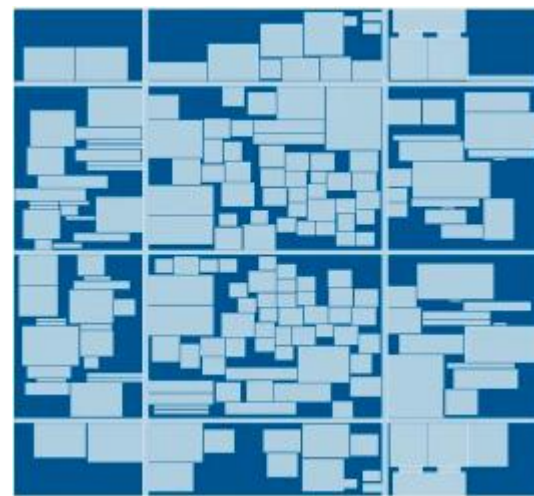
- 布局需要解决上亿个基本模块的放置问题
- 若假设布局范围是 1000×1000



总状态数量: $\sim 10^{123}$



总状态数量: $\sim 10^{360}$



总状态数量: $> 10^{100,000}$

Google的Alpha GO使用**176个**
GPU训练**40天**才学会下围棋

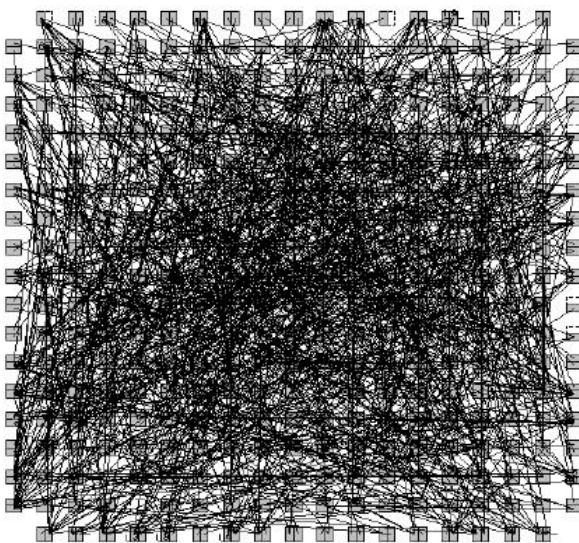
■ 什么是布局

□ 好的布局结果 对比 差的布局结果

– 230个基本模块布局到FPGA上

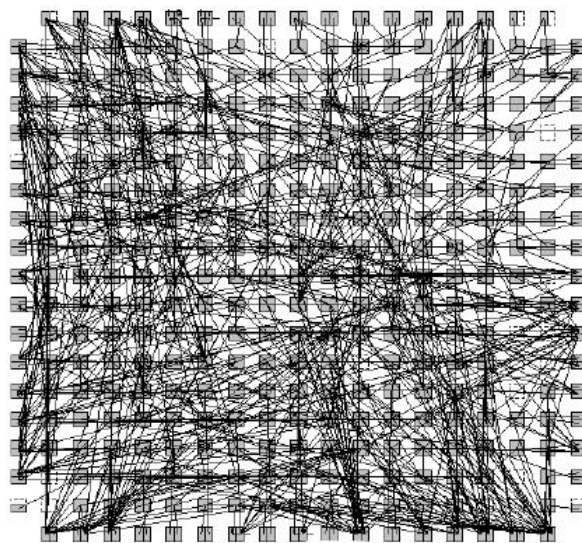
➤ 二者总的线长差距达十倍，因此好的布局很重要

随机生成的布局结果



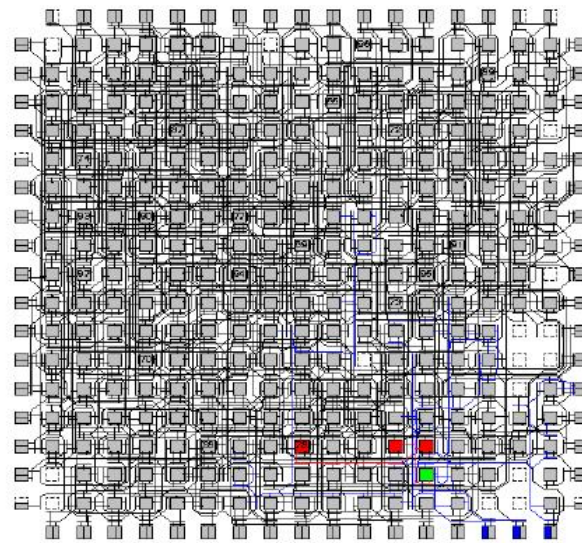
WL = 5.47e+4

优化的布局结果



WL = 6.73e+3

最终布线结果

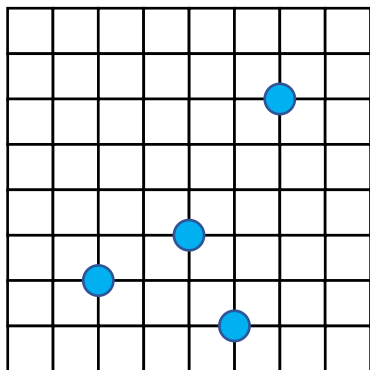


■ 什么是布局

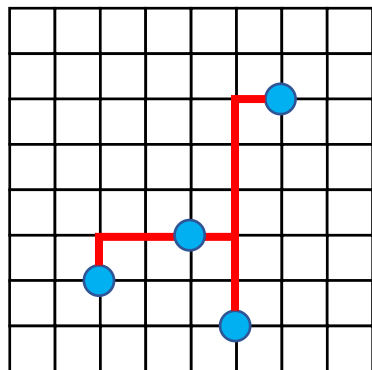
□ 怎样计算总线长

– 一般如何进行布线，就如何预估总线长

➤ 有以下三种曼哈顿距离的线长预估模型

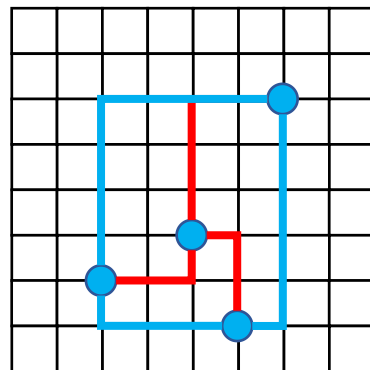


假设用一个线网连接4个引脚

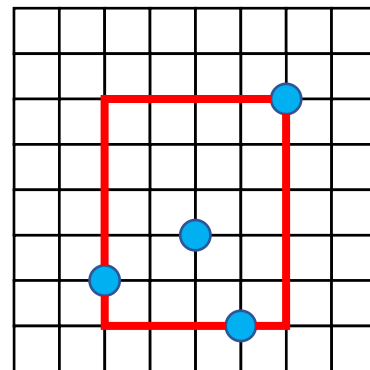


最小steiner树模型

$$\sum |x_i - x_j| + \sum |y_i - y_j| \quad \max |x_i - x_j| + \max |y_i - y_j|$$



团(clique)模型



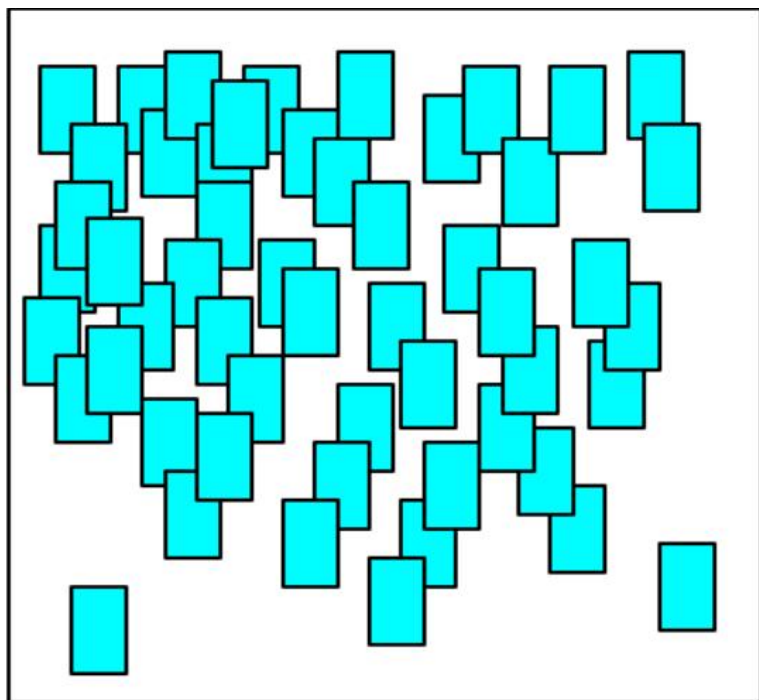
半周长(HPWL)模型

属于曼哈顿距离模型

■ 布局算法历史

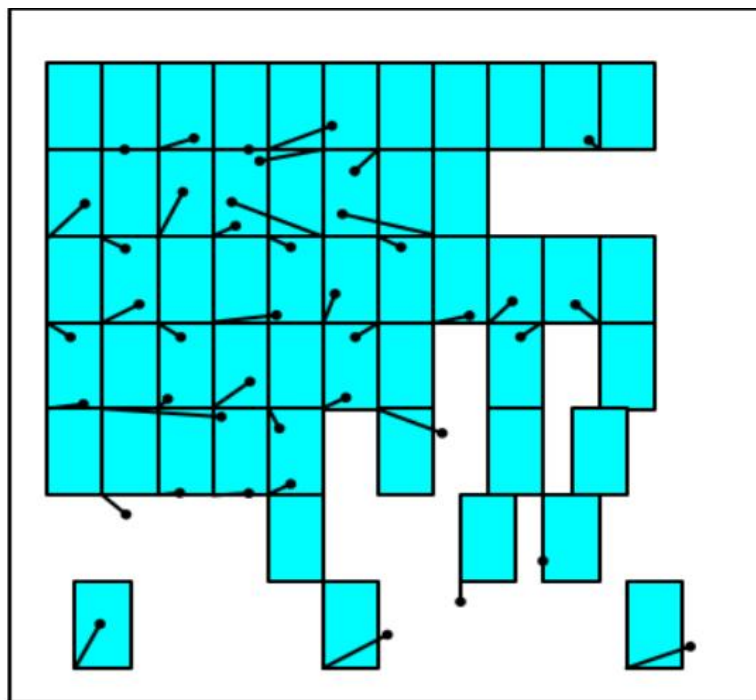
□ 芯片布局一般有三个流程

全局布局阶段



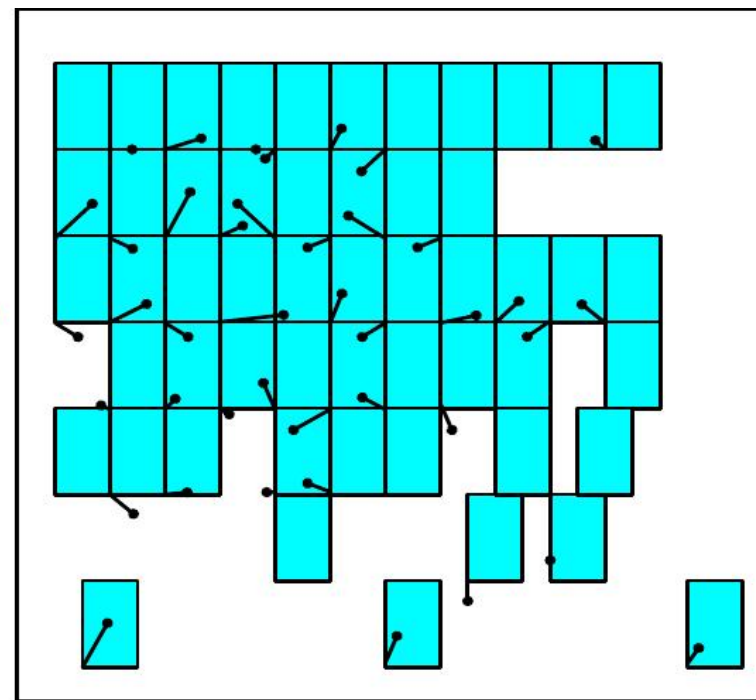
总线长: 10^7

合法化阶段



总线长: 1.05×10^6

详细布局阶段



总线长: 9.8×10^5

■ 布局算法历史

1970-1980	1980-1990	1990-2010			>2010	
基于划分	模拟退火	最小割 (多层次)	解析式(analytic)		解析式	
			二次型 (Quadratic)	非线性 (Nonlinear)	二次型	非线性
Breuer	Timberwolf VPR	FengShui	GORDIAN	APlace	POLAR	ePlace Replace
Dunlop & Kernighan	Dragon	Capo	BonnPlace	Naylor Synopsis	SimPL ComPLx	DREAMPlace
Quadratic Assignment		Capo+Rooster	mFar	NTUplace	MAPLE	
Cadence QPlace			Kraftwerk	mPL6		
			FastPlace			
			Warp3			

PART 02

模拟退火

■ 模拟退火

- 介绍两种问题建模方式
 - Timberwolf^{[1][2]}
 - Dragon^[3]
- 都使用模拟退火算法来解决布局问题
 - 但采用不同的问题建模方式

[1] C. Sechen, A. Sangiovanni-Vincentelli, "The TimberWolf placement and routing package," JSSC, 1985.

[2] C. Sechen, A. Sangiovanni-Vincentelli, "TimberWolf3.2: A new standard cell placement and global routing package," DAC, 1986.

[3] Maogang Wang, Xiaojian Yang, M. Sarrafzadeh, "Dragon2000: standard-cell placement tool for large industry circuits." ICCAD, 2000.

■ 模拟退火

□ 问题定义

– 输入

- 网表 N , 包含顶点集 V 和超边集 E
- 每个模块 v_i 的面积 A_i 、宽 w_i 和高 h_i

– 输出

- 模块的布局坐标 (x_i, y_i)
- 要求模块之间不能重叠, 且所有模块均在给定的布局范围内
- 最小化总线长
 - 一般是半周长线长预估模型HPWL

■ 模拟退火——Timberwolf

□ Timberwolf 布局过程分为两个阶段

– 阶段一

- 优化目标是**线长**
- 允许模块跨行移动、交换
- 当温度 T 降低到特定数值，开始第二阶段的退火过程

– 阶段二

- 仅允许同一行的，相邻的模块进行交换和移动
- 进一步优化布局的总线长

– 两个阶段最大的区别是生成新状态的策略不同

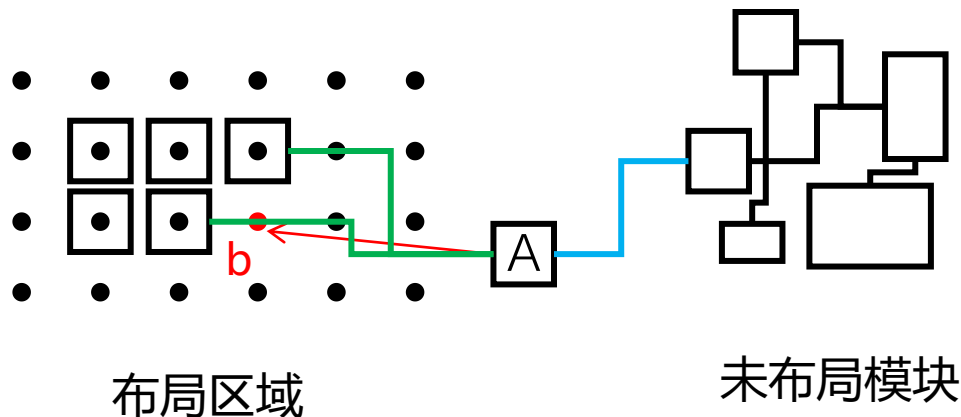
■ 模拟退火——Timberwolf

□ Timberwolf 初始化布局

– 贪心策略

- 简单的策略，依次选模块，然后选择位置进行布局
- 选择一个未布局的模块A，这个模块A与已经布置的模块之间的连接密度最大
- 选择一个位置，模块A布局在这个位置上的代价(目标函数)最小
- 简单但十分高效，非常适合生成初始化布局方案

模块A与已布局的模块之间
连接关系为2
是剩余未布局模块中最大的

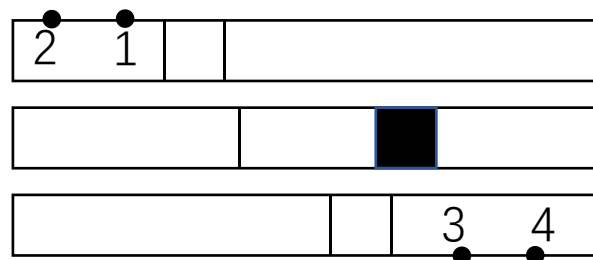
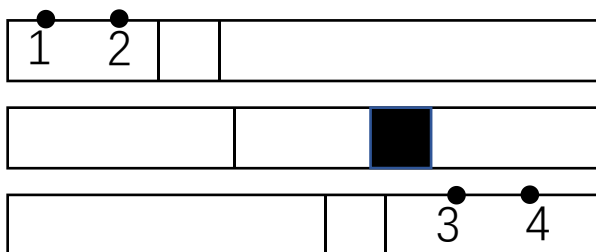
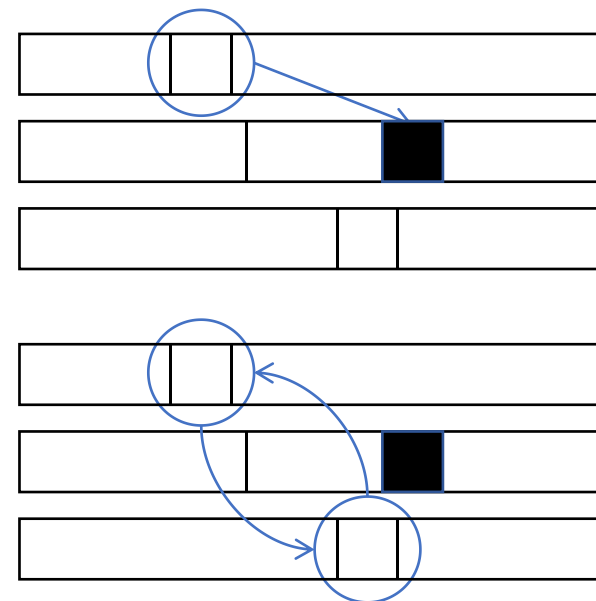


放置在位置b处，是其他候选位置中，预估线长最短的

■ 模拟退火——Timberwolf

□ Timberwolf 第一阶段采用三种移动步

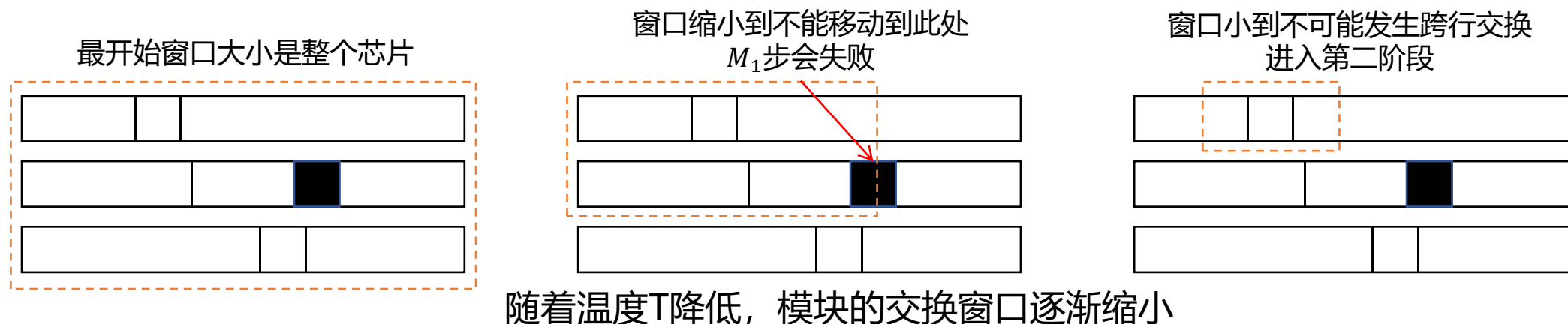
- M1: 移动模块到一个新的位置
- M2: 交换两个模块
- M3: 改变模块的朝向



■ 模拟退火——Timberwolf

□ Timberwolf 三种移动实现细节

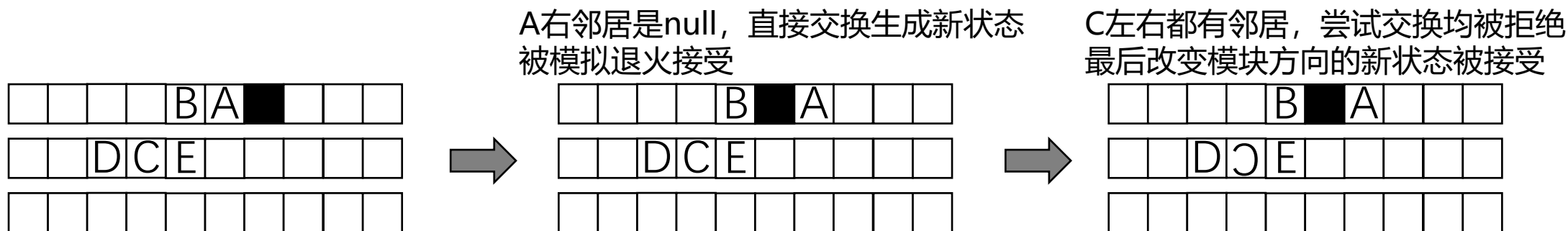
- 只有M1和M2两种移动策略会被选择且 $P(M_1) = \frac{4}{5}$, $P(M_2) = \frac{1}{5}$
- 当M1失败时, 尝试用M3策略产生新状态
- 移动和交换的距离有窗口限制
 - 初始化移动和交换的窗口大小 R 能够完全覆盖整个芯片
 - 随着温度 T 下降逐渐缩小窗口, 收缩策略 $R \propto \log T$
 - 当窗口缩小到不可能发生跨行交换或移动时, 进入第二阶段



■ 模拟退火——Timberwolf

□ Timberwolf 第二阶段采用的移动步

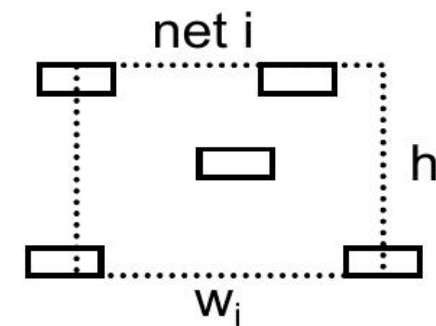
- 相邻模块交换——生成模拟退火的新状态
 - 按以下步骤依次生成新状态，如果新状态没有被接受，则继续尝试下一步
 - 随机选择一个模块 A
 - 在行为主的芯片上，模块会尝试与左右邻居进行交换(列为主的芯片，就上下邻居)
 - 如果左右邻居存在Null
 - 如果左邻居 L 是Null，就尝试交换 A 和 L
 - 如果右邻居 R 是Null，就尝试交换 A 和 R
 - 如果左右邻居均存在，则随机选一个尝试交换。交换失败就选另一个邻居
 - 若仍然失败，则尝试改变模块的朝向



■ 模拟退火——Timberwolf

□ Timberwolf 的代价函数(cost function)由三部分组成

- 第一部分，线长预估： $C_1 = \sum (\alpha_i w_i + \beta_i h_i)$
 - 其中 α_i 和 β_i 是水平布线和垂直布线的权重
 - 对于关键线网，增加 α_i 和 β_i
 - 这种代价函数有利于适应水平布线和垂直布线代价不同的情况
- 第二部分，重叠惩罚： $C_2 = \sum_{i \neq j} (O(i, j) + \alpha)^2$
 - 其中 $O(i, j)$ 是重叠量统计函数
 - $\alpha = 3$ ，保证重叠惩罚优先级高于行长度惩罚
- 第三部分，行长度惩罚： $C_3 = \sum_r \beta |l(r) - d(r)|$
 - 其中 $l(r)$ 表示当前行 r 所有模块长的总和，包括模块间空白区域， $d(r)$ 表示行 r 预期长度
 - $\beta = 5$ 。目的是均衡各个区域模块数量，缩小空白区域



□ 第二阶段仅保留代价函数 C_1

■ 模拟退火——Timberwolf

□ Timberwolf 退火策略

– 温度降低策略: $T_k = r(k)T_{k-1}, k = 1, 2, 3, \dots$

– 在每个温度上, 内循环 $n \times m$ 次

- 其中 n 代表模块数量
- m 是用户输入的超参数

当 $T \geq$	$r(k)$
40000	0.80
20000	0.84
10000	0.88
5000	0.91
200	0.94
100	0.90
50	0.85
5	0.80
1.5	0.70
0	0.10

■ 模拟退火——Timberwolf

□ 总结——Timberwolf

– 优势

- 简单的策略，简单的步骤

– 缺点

- 可拓展性不高，面对更大规模的问题，需要更多的内循环次数以充分探索解空间
- 内循环次数参数 m 是人为设定的，未必是质量和效率的最佳平衡点

■ 模拟退火

□ 问题定义

– 输入

- 网表 N , 包含顶点集 V 和超边集 E
- 每个模块 v_i 的面积 A_i 、宽 w_i 和高 h_i
- **一个初始布局**

– 输出

- 模块的布局坐标 (x_i, y_i)
- 要求模块之间不重叠, 且所有模块均在给定的布局范围内

■ 模拟退火——Dragon2000

□ Dragon2000 采用划分+模拟退火

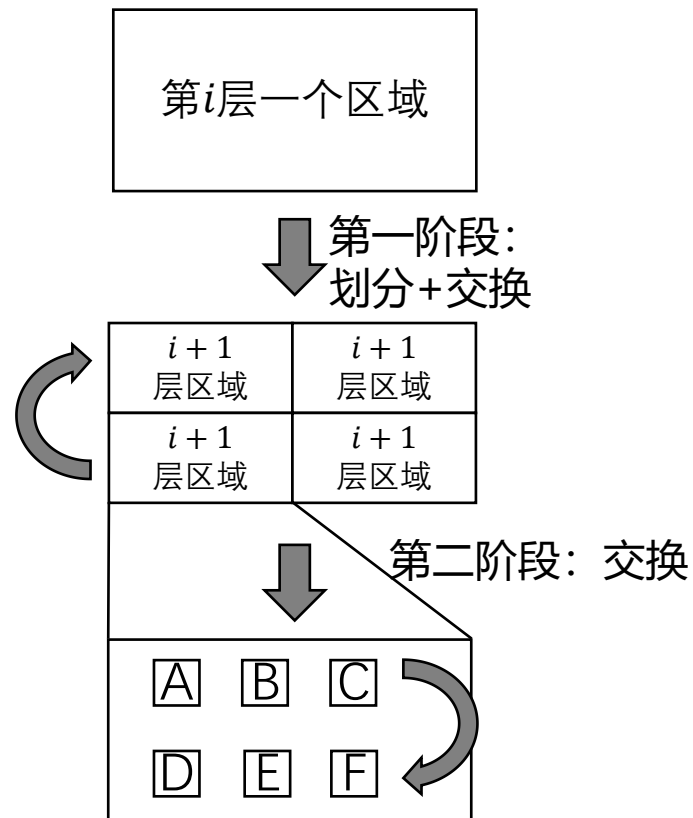
– 第一阶段

- 当 $i = 1$ 时，划分对象是整个芯片
- 第 i 次划分的区域 h_i 继续划分为4个子部分 h_{i+1}
- 对 $i + 1$ 层的 h_{i+1} 子部分集合使用模拟退火算法来优化线长
 - 注意：交换的是两个区域包含的所有模块，不是某一个特定模块

– 第二阶段：当 h_{i+1} 内部的模块数量少于7个时停止划分

- 再对 h_{i+1} 区域内部的模块位置用模拟退火进行优化

– 第三阶段：基于贪心策略进行详细布局以优化结果

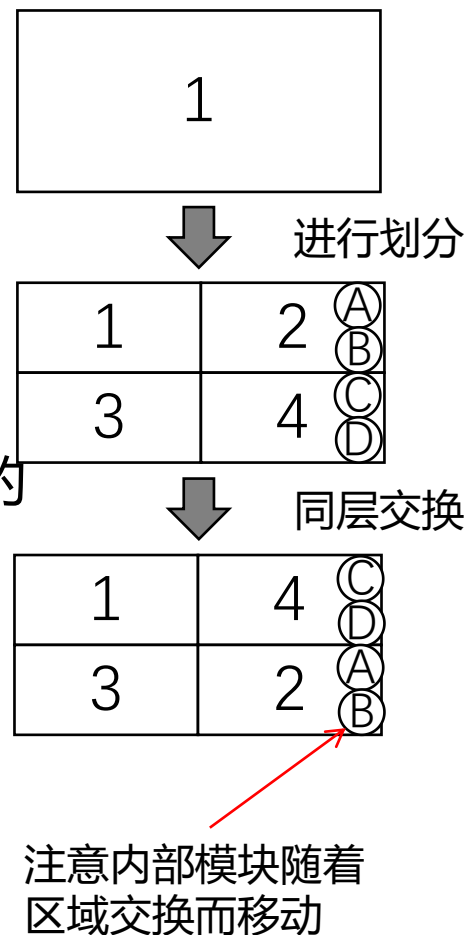
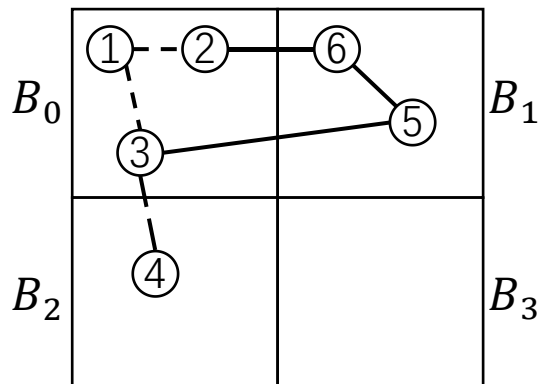


■ 模拟退火——Dragon2000

□ Dragon2000 自顶向下的划分方法

– 第一阶段——层次划分

- 把第 i 层的大区域划进一步分成4个子区域得到第 $i + 1$ 层
- 在第 $i + 1$ 层的 4^{i+1} 个子区域之间尝试交换
- Dragon2000测试了四种不同的划分策略，选择其中效果最好的
 - 我们以下图连接情况为例说明Dragon2000实验的划分策略
 - 例子包含三条超边 $\{3,4\}$ 、 $\{1,2,3\}$ 和 $\{2,3,5,6\}$



■ 模拟退火——Dragon2000

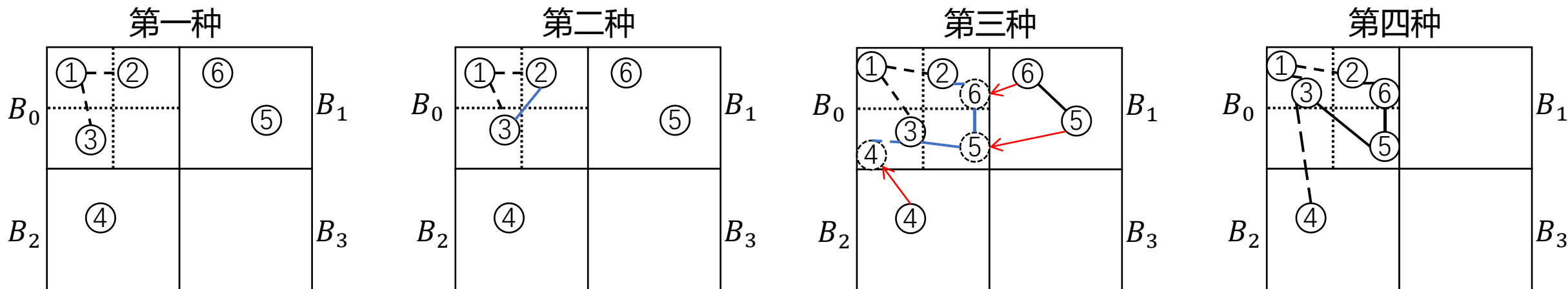
□ Dragon2000 测试多种划分策略，以划分 B_0 为例

– 多种不同策略

- 第一种：去掉存在外联关系的超边(连接不在 B_0 的顶点，就去掉整条超边)
- 第二种：仅去掉超边中不在 B_0 的顶点，对比第一种，(2,3)连接关系保留
- 第三种：在第二种的基础上引入不可移动的虚拟顶点(当时常用的算法实现)
- 第四种：允许单独模块进行区域流动(例如⑤⑥移动到区域 B_0)

超边一：{3,4}
超边二：{1,2,3}
超边三：{2,3,5,6}

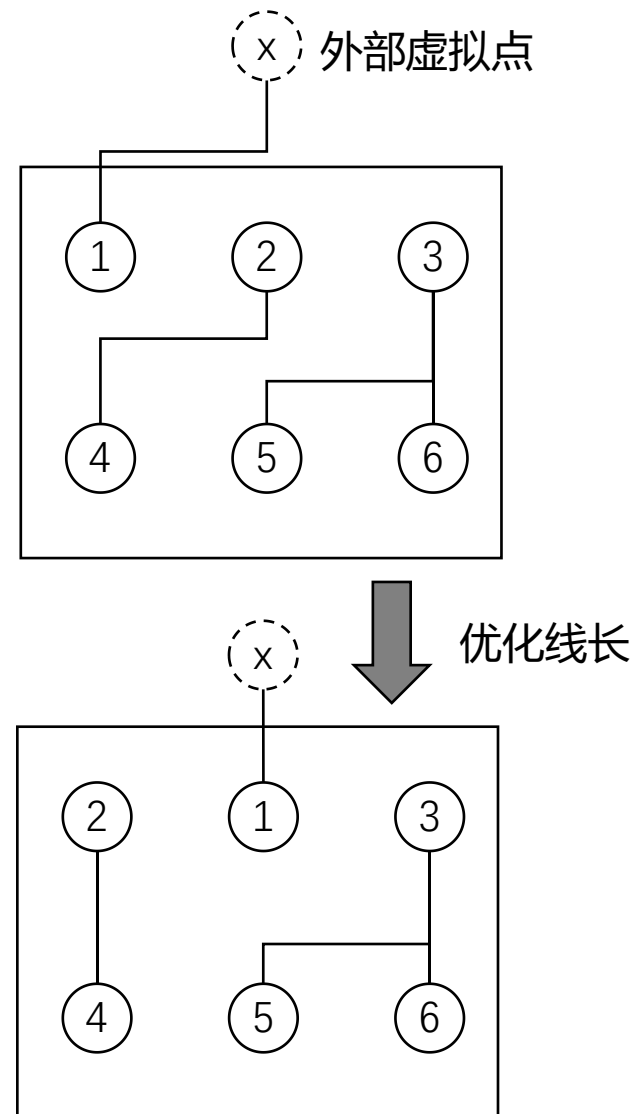
– 实验发现第二种策略效果最好，打破一贯的认知



■ 模拟退火——Dragon2000

□ Dragon2000 第二阶段

- 若第 i 层的子区域内部模块数量少于7个时停止划分，进入第二阶段
- 执行低温度下的模拟退火
 - 交换子区域内部的模块位置
- 实验发现可以进一步获得5%~7%的线长提高

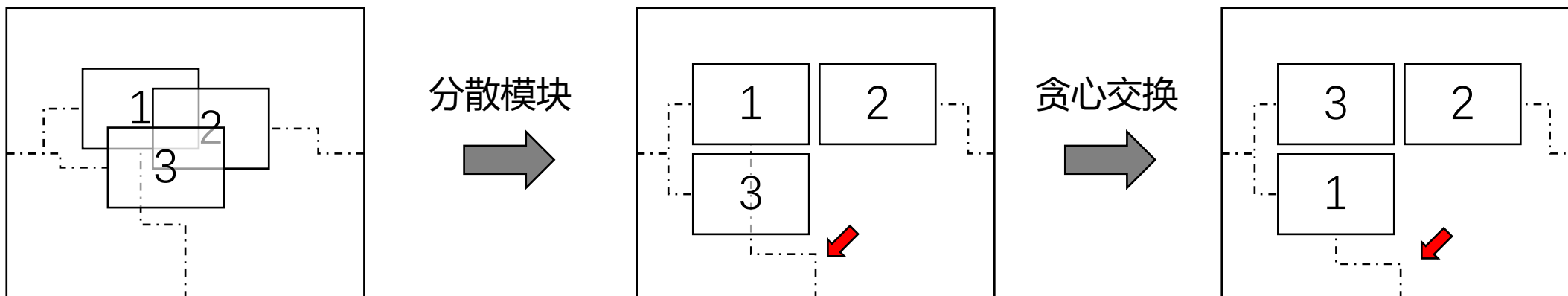


■ 模拟退火——Dragon2000

□ Dragon2000 第三阶段

– 采用贪心策略进行详细布局优化

- 第一步，分散所有存在重叠的模块
- 第二步，随机选择一个模块
 - 选择一个垂直或者水平方向进行交换
 - 如果交换后能够获得线长收益则执行交换
 - 下图中交换12的线长是负收益，但交换13是正收益的



■ 模拟退火——Dragon2000

□ 总结——Dragon 2000

- 也是基于模拟退火的方案，但比TimberWolf商业版快1.9倍
 - 划分策略缩小问题规模
 - 逐步细化划分，提高布局结果质量
- 同时和TimberWolf得到的布局结果在线长指标上不相上下
- 在更大规模的芯片上效果更好

■ 模拟退火

□ 总结

– 优势在于

- 对不同初始状态适应性好
- 解质量对比同时期其他工具好，Timberwolf曾作为一段时期内的商业工具方案
- 相对实现容易

– 不足之处在于

- 模拟退火还是太慢了，对比同时期其他方案
- 模拟退火的可拓展性不高，不能适应更大规模的布局问题求解

PART 03

图划分

■ 图划分——CAPO

□ 基于图划分的布局算法

- 图划分算法在前面的章节已经介绍过
 - Fiduccia-Mattheyses
 - 多层次划分, 例如 hMETIS
- 有两个开源的学术级布局工具
 - CAPO^[1]
 - Feng-shui^[2]

[1] A.E. Caldwell, A.B. Kahng, I.L. Markov, "Can recursive bisection alone produce routable, placements?," DAC, 2000.

[2] Ameya R. Agnihotri, Satoshi Ono, Patrick H. Madden, "Recursive bisection placement: feng shui 5.0 implementation details," ISPD, 2005.

■ 图划分——CAPO

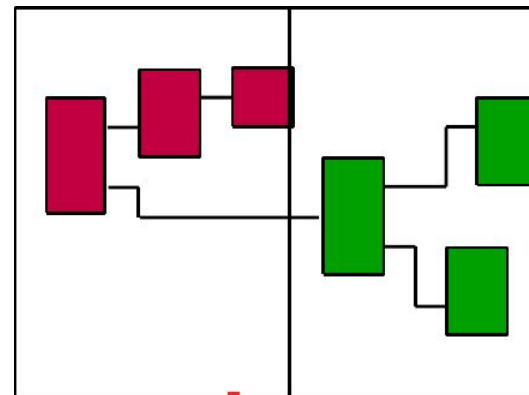
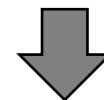
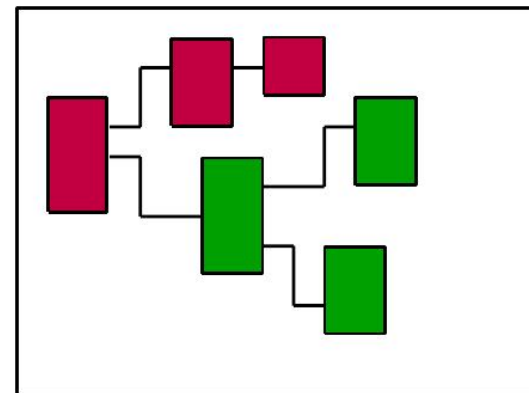
□ 问题定义

— 输入

- 模块集合 $V = \{v_1, v_2, \dots, v_n\}$, 超边 E
- 模块 v_i 的面积 A_i 、宽 w_i 和高 h_i

— 输出

- 一种划分结果 $\{X_1, X_2, \dots, X_n\}$
- 要求 X_i 集合内部模块面积之和与 X_j 集合内部模块面积之和大致相等
- 最小化割代价。过长的边会经过多个划分边界，不利于割代价的最小化
- 芯片区域的划分与网表划分一同进行，网表划分后模块位置也会一同确定

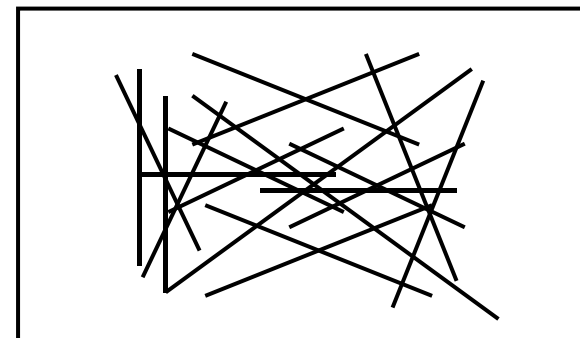


■ 图划分——CAPO

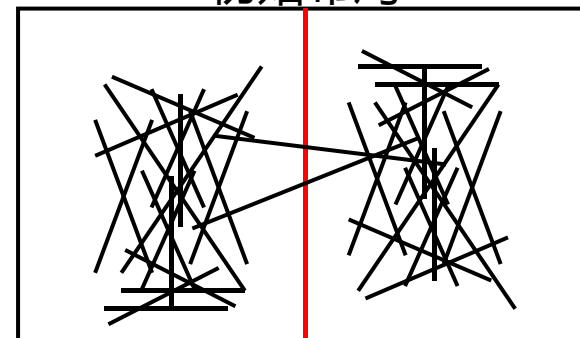
□ 算法流程——递归执行

- 聚集互相连接的模块，找到初始划分方案
- 执行图划分算法，划分之间的割代价最小
 - 注意：芯片布局范围也一同被划分
 - 划分算法有许多变种，如不同的代价函数、不同的划分算法细节、不同的划分策略等等
- 每个子电路放置到一个具体的布局范围内
 - 注意，仅划分算法为多路划分时需要考虑布局位置
 - 而CAPO采用的二路划分方案，不需要指定布局位置。因为划分X放左边还是放右边，不会影响线长

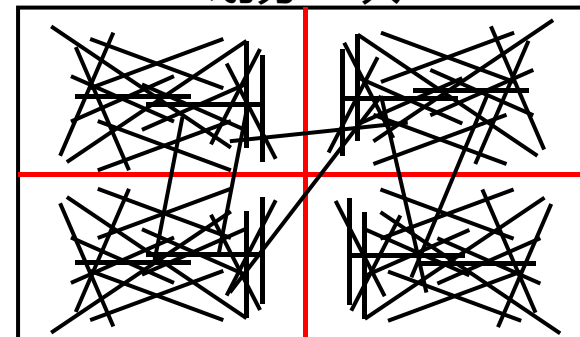
□ 这种算法也被称为最小割布局算法



初始布局



划分一次



划分两次

■ 图划分——CAPO

□ 算法细节

– 划分算法实现框架

- 若模块数量超过200个，使用多层次划分算法
- 若模块数量在35到200之间，使用FM算法^[1]
- 若模块数量少于35个，使用分支界限法(branch-and-bound)

– 划分算法实现细节

- 容忍策略：增加模块的移动概率
 - 该策略允许划分不遵守二等分的约束
- 重划分策略：同一个初始状态开始以不同容忍度，多次调用划分算法
- 动态容忍策略：拥有更多空白比例(whitespace)的区域在划分时拥有更高容忍度
 - 空白比例指当前划分区域内未利用芯片面积与总的芯片面积的比值
 - 意思是当前区域内未利用芯片面积越多，那么进行划分时越不需要遵守等分的约束

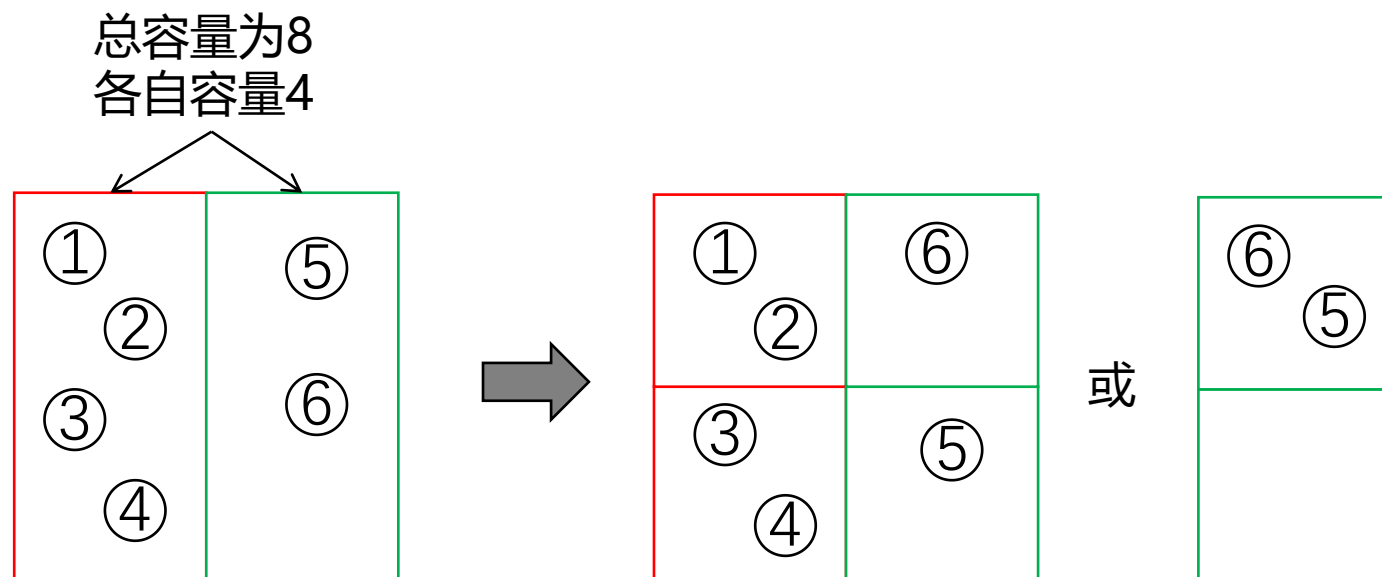
[1] C.M. Fiduccia, R.M. Mattheysees, "A Linear-Time Heuristic for Improving Network Partitions," DAC, 1982.

■ 图划分——CAPO

□ 基于空白比例的动态容忍策略

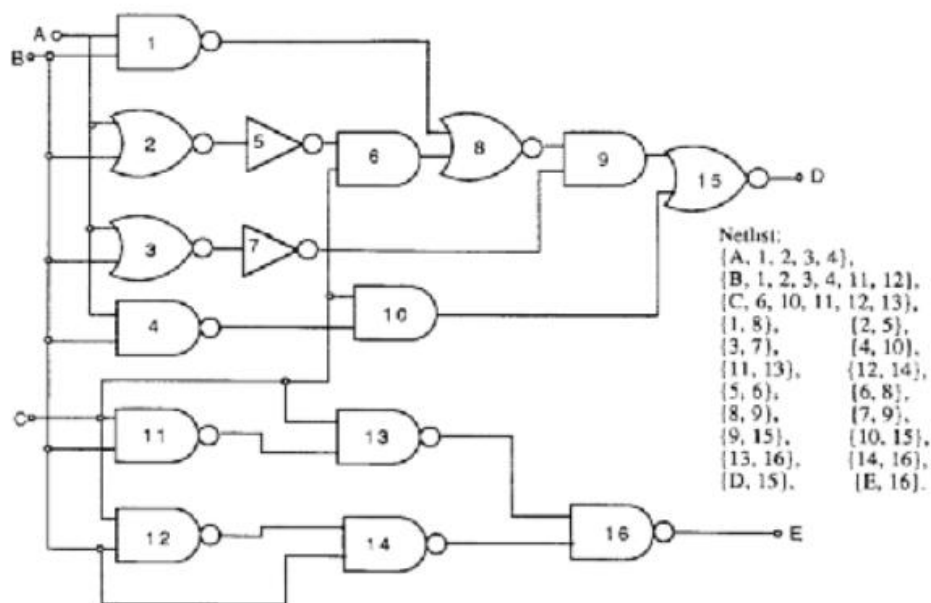
– 假设下图的每个子区域能容纳4个模块

- 红色区域的划分容忍度就是0，不均衡的划分就会导致布局失败
- 绿色区域的划分容忍度是 ∞ ，即使所有模块在一个划分内也能成功布局

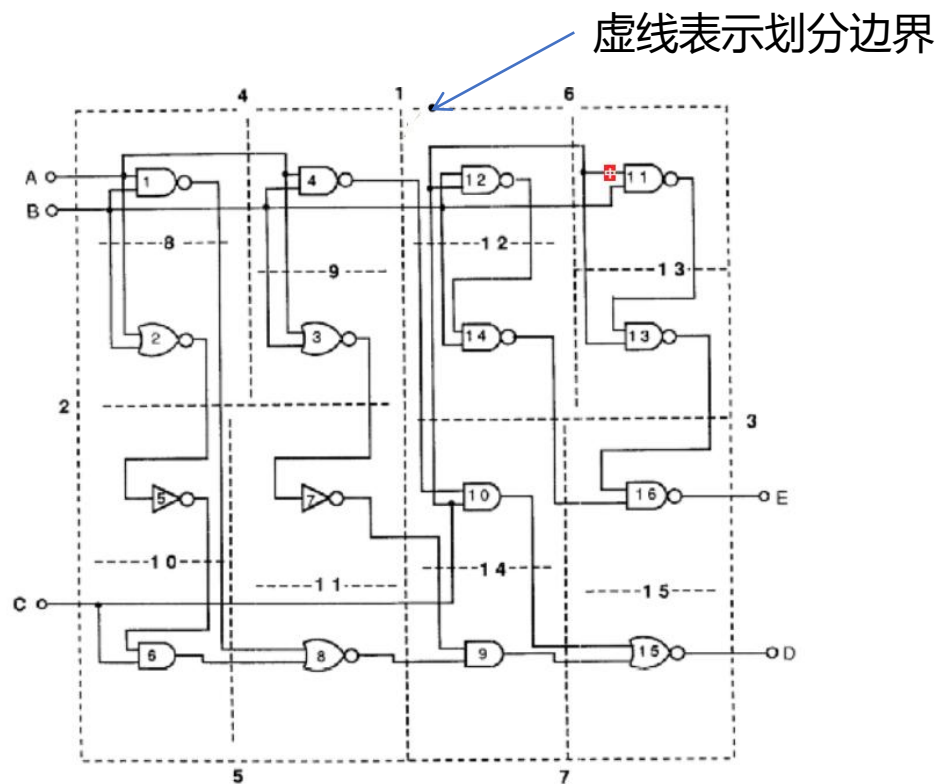


■ 图划分——CAPO

- 算法一直执行，直到每个模块都有合法的布局位置
 - 例如右图每个划分内部最终只有一个模块，此时算法退出



芯片

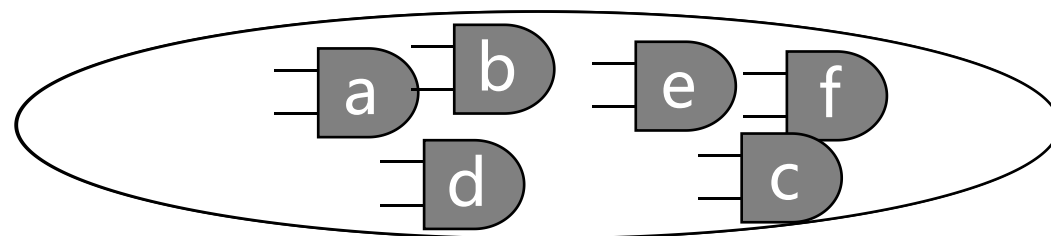
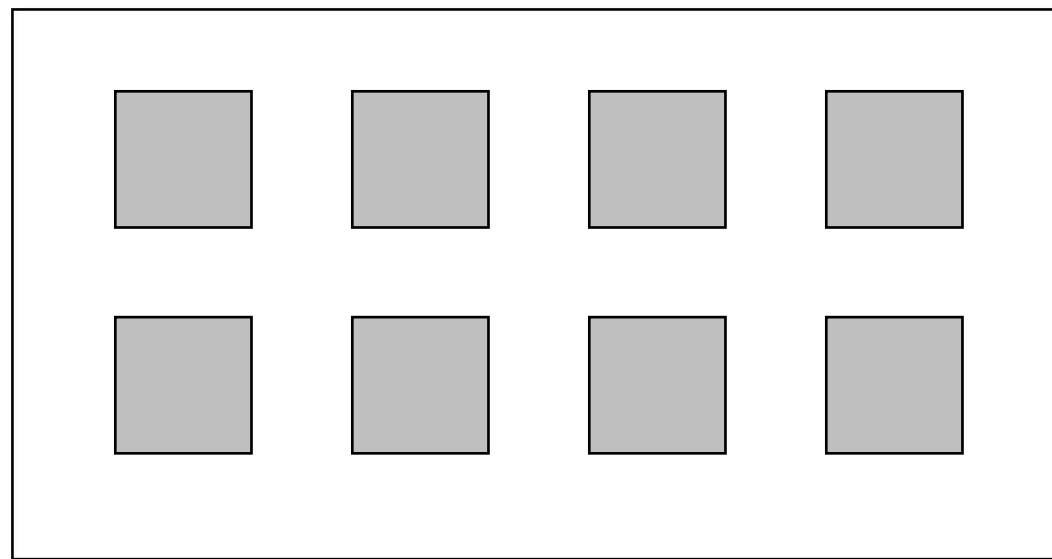
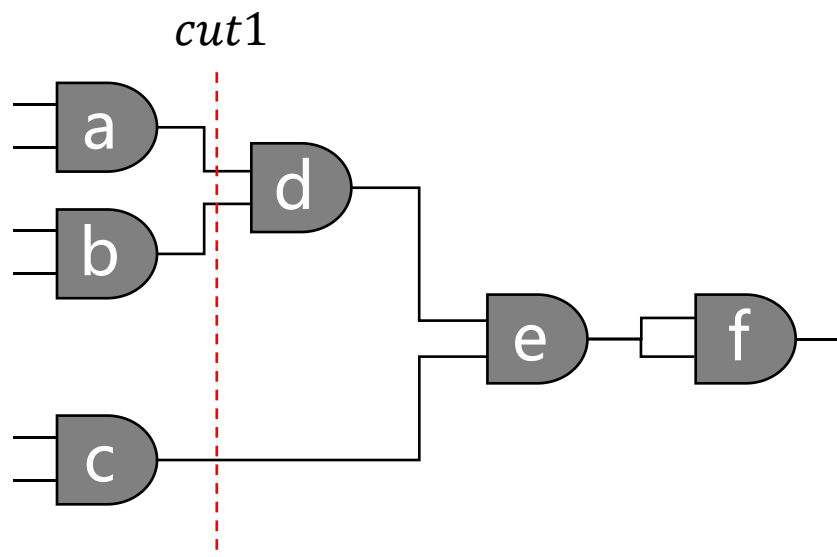


布局结果

■ 图划分——CAPO

□ 举例

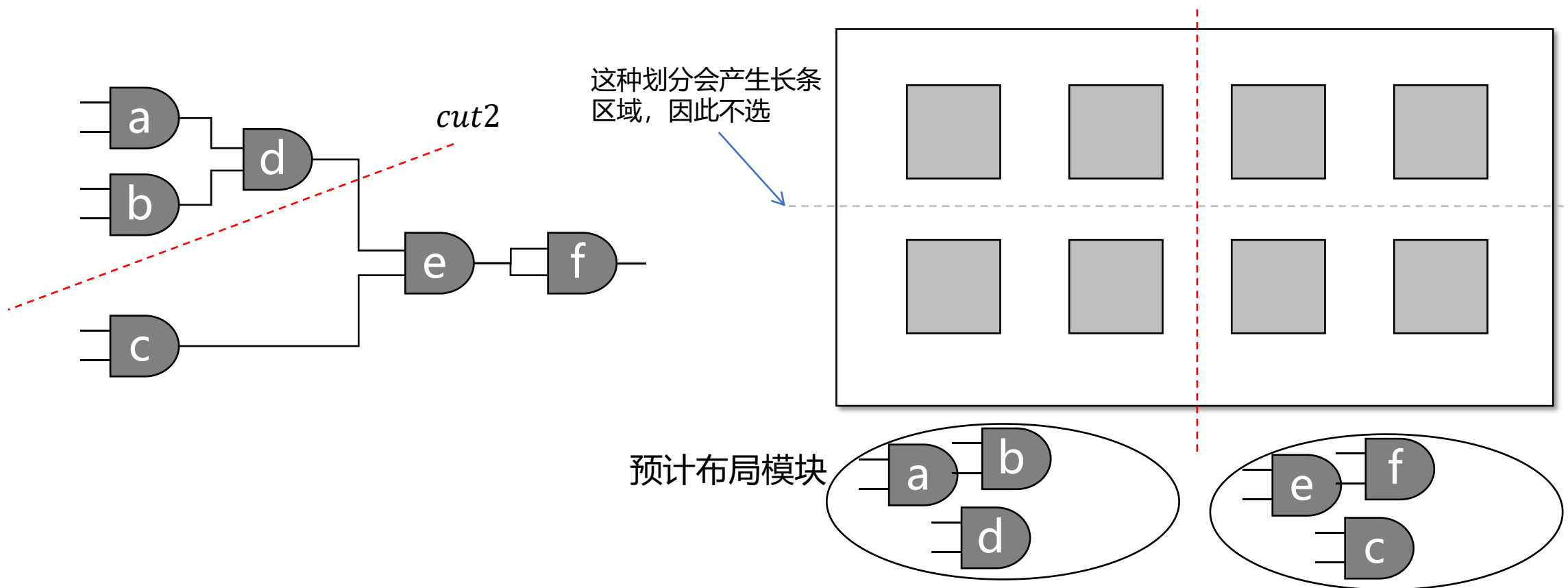
- 左图所示的网表，要布局到右图所示的芯片区域中
- 输入网表、芯片和初始划分 $cut1$



■ 图划分——CAPO

□ 举例

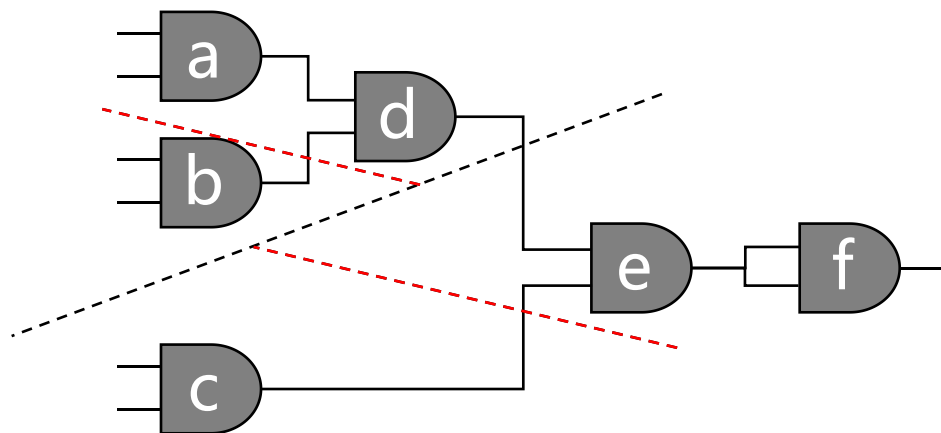
- 利用FM算法对初始划分进行优化，得到新划分 $cut2$
- 注意对芯片区域也进行划分，要尽量保证划分后的区域不是一个长条



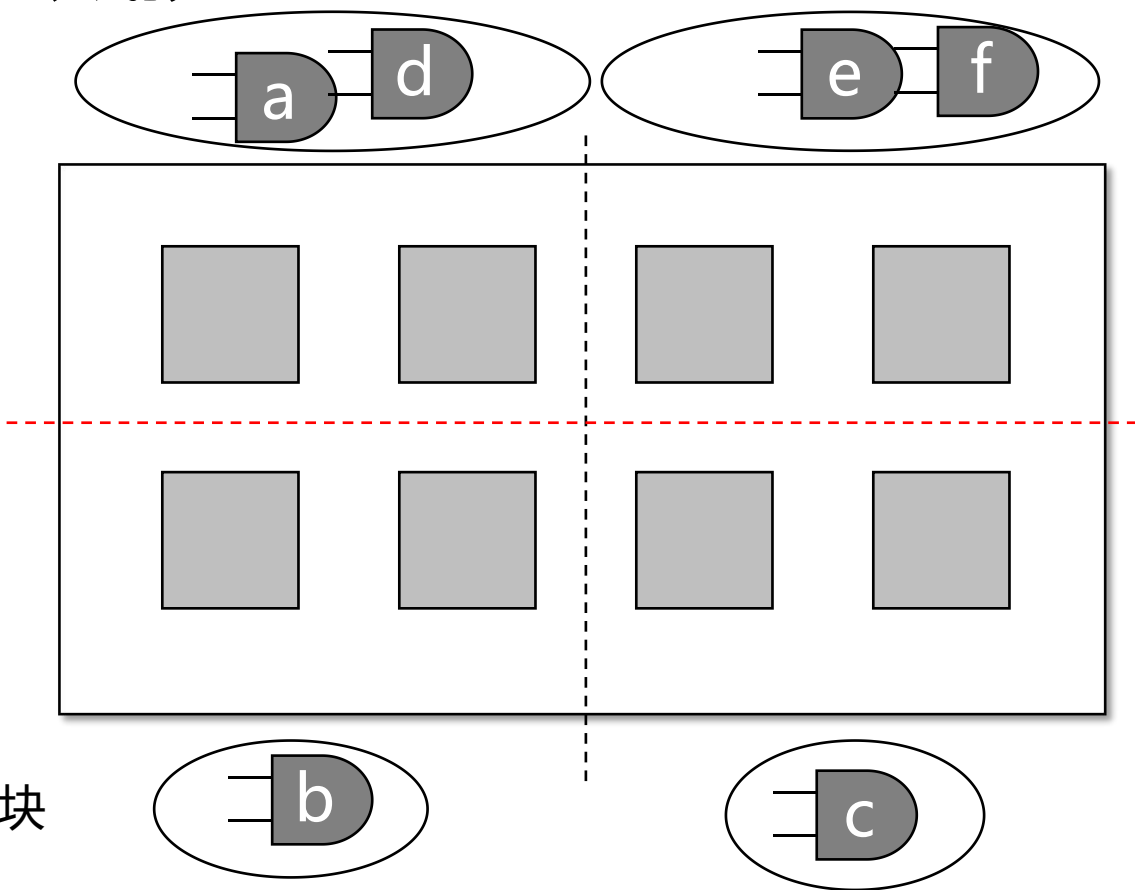
■ 图划分——CAPO

□ 举例

- 对 $\{a, b, d\}$ 和 $\{c, e, f\}$ 的初始划分进行二次划分
- 芯片区域也进行划分



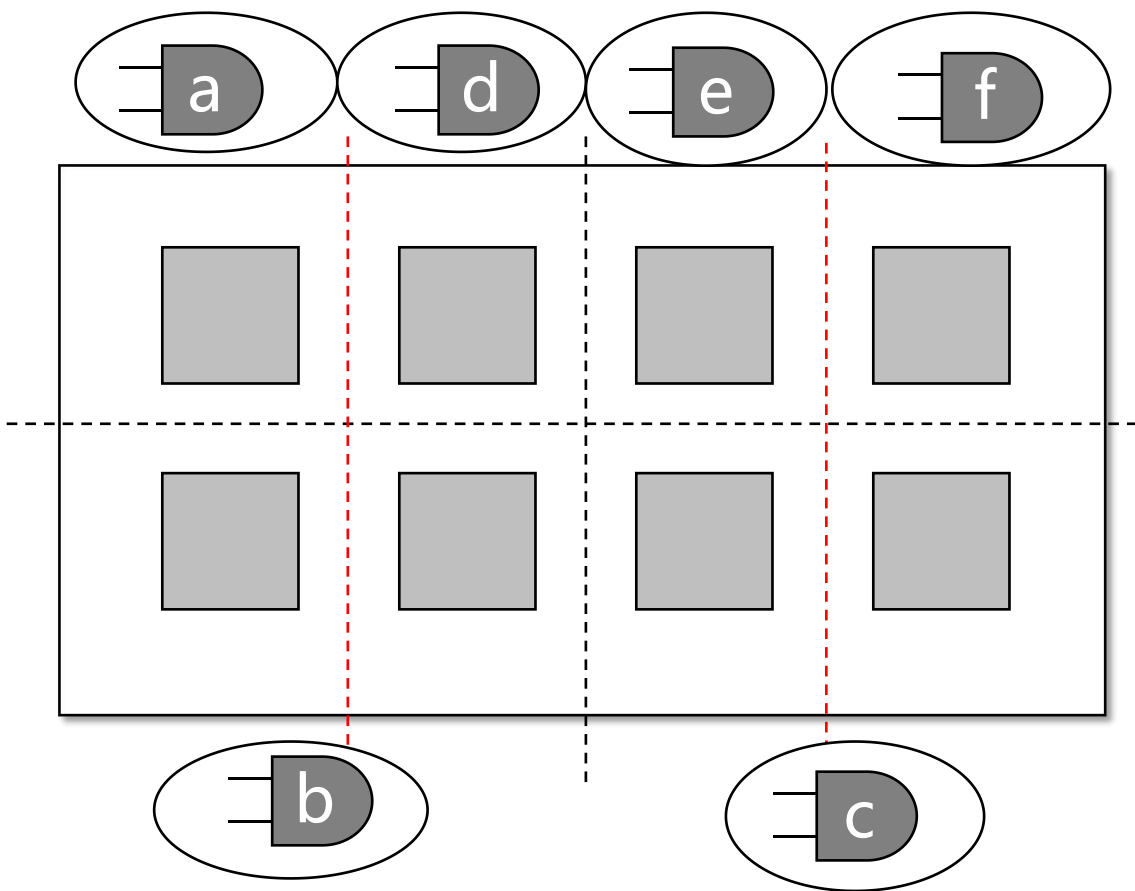
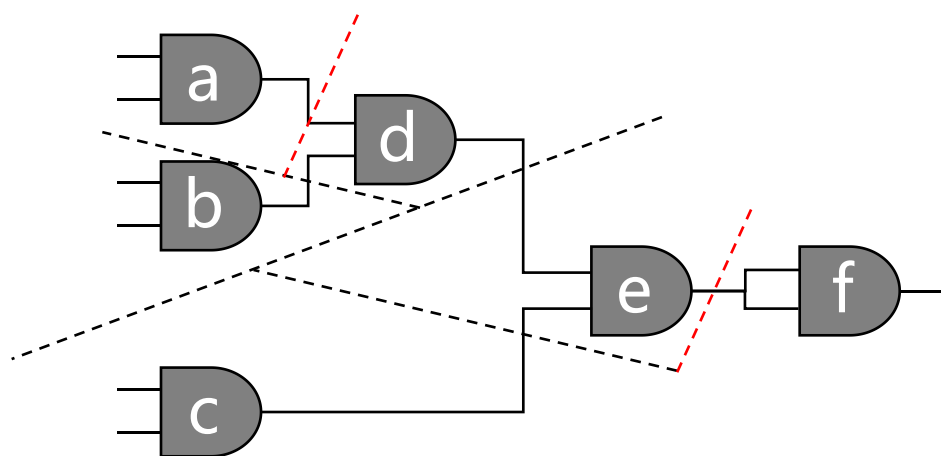
预计布局模块



■ 图划分——CAPO

□ 举例

- 对 $\{a, d\}$ 和 $\{e, f\}$ 的二次划分进行划分
- 芯片区域也进行划分

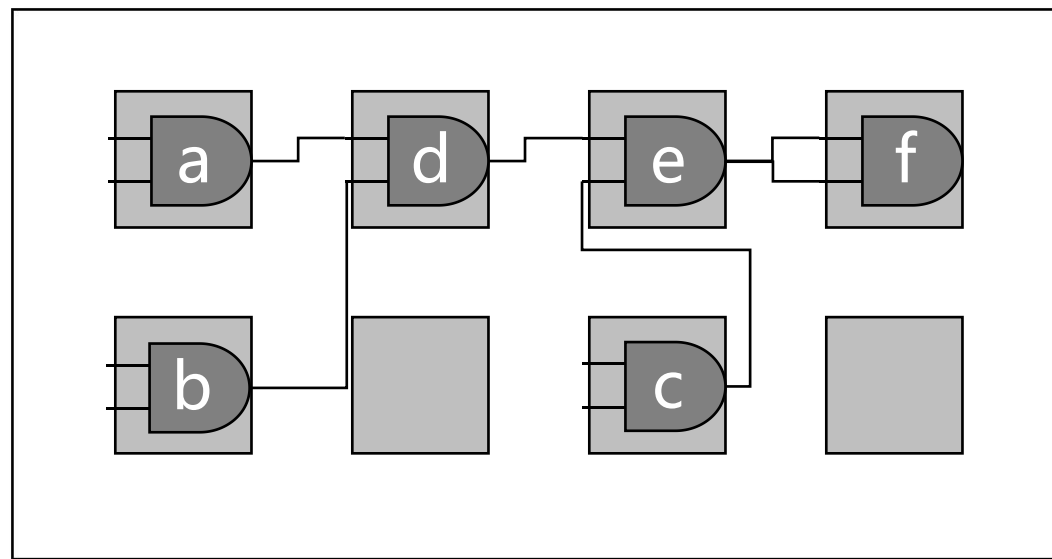
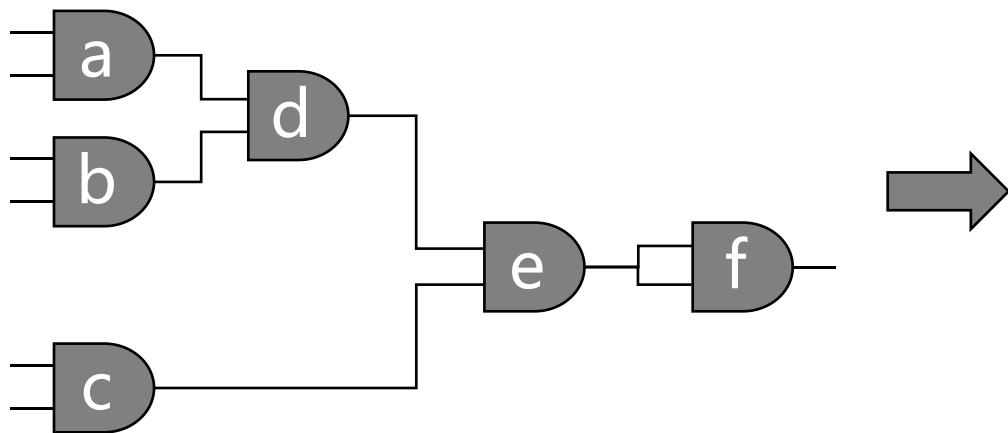


对于模块b和模块c来说，允许划分是不均衡的

■ 图划分——CAPO

□ 举例

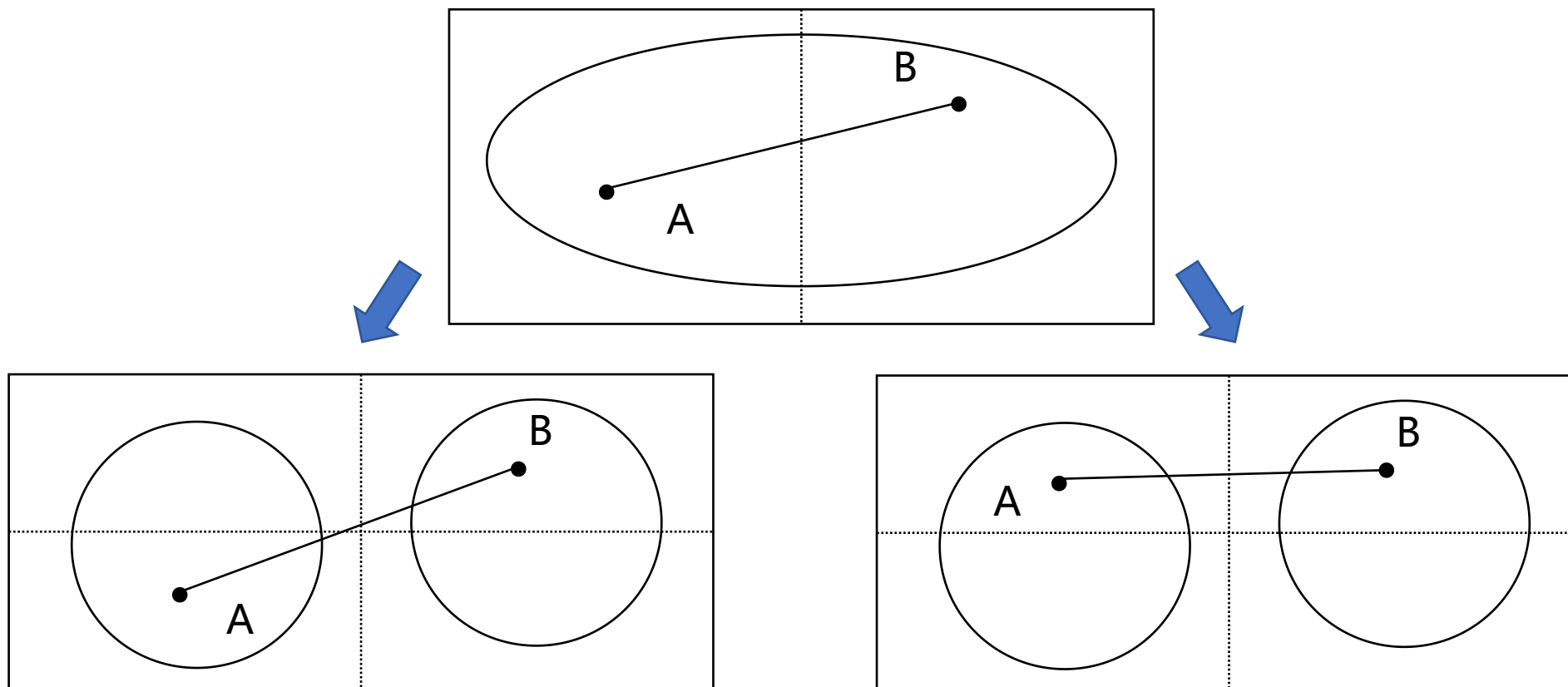
- 此时每个区域仅包含一个模块
- 布局芯片，然后算法退出



■ 图划分——CAPO

□ 外部端点连接问题

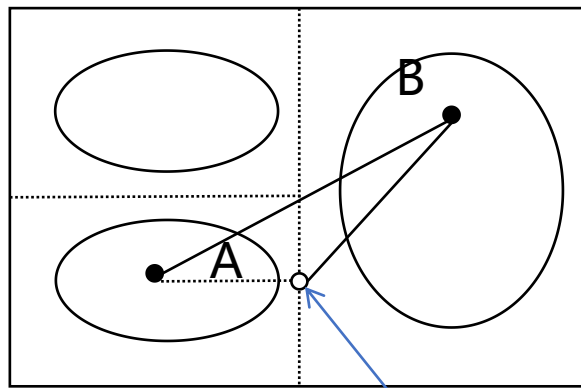
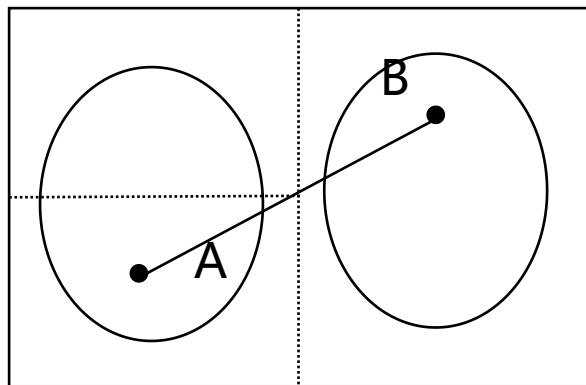
- 过去的划分算法没有考虑到外部端点的相对位置
 - 以下两种划分产生的总线长是不同的，右边的较短



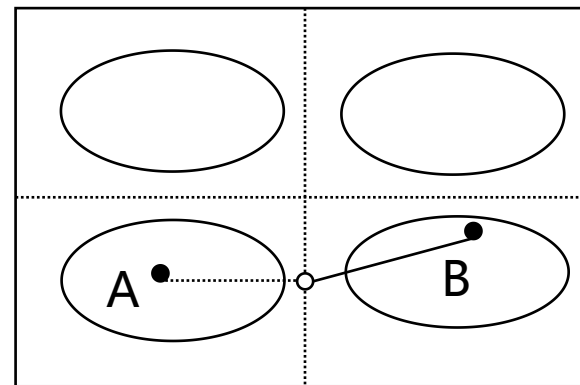
■ 图划分——CAPO

□ 外部端点连接问题

- 需要考虑线网连接外部端点的位置信息
- 在划分边界上增加虚点(dummy terminal)^[1]
 - 虚点会尝试把B拉到同一侧
 - 具体做法是如果B不在虚点一侧，那么总的割代价+1



虚点



[1] A.E. Dunlop, B.W. Kernighan, "A Precedure for Placement of Standard-Cell VLSI Circuits," TCAD, 1985.

■ 图划分——CAPO

□ 总结——CAPO

- 只能处理标准模块的放置问题
 - 不支持异构模块的放置问题，例如放置一个大型的宏模块，或者对放置模块的区域有约束
- 完全基于递归二划分的布局算法
 - 使用一些小的启发式方法提高划分的质量
- 获得较好的布局结果主要是因为：
 - 过去几年中对于多层次划分算法的提高
 - 在划分的具体实现细节上下功夫

■ 图划分

□ 总结——基于划分布局算法

– 优点

- 算法运行非常快
- 布局质量较好
- 随问题规模增长呈线性时间复杂度

– 缺点

- 不容易实现，要细致调整划分约束以防止布局失败
- 对复杂约束支持不友好，例如布局区域限制、关键路径线长约束等
- 解的质量不稳定

感谢！
